University of Moratuwa
Department of Electronic and Telecommunication Engineering

UART Implementation in FPGA

Group 38:

230726L Wijesinghe U. G. S. K. D.
230727P Wijesinghe W. A. P. W.

EN 2111 - Electronic Circuit Design

May 1, 2026

Contents

1 Introduction **1**

2 Verilog RTL Code **1**

2.1 Transceiver module 1

2.2 Test bench module 6

2.3 Top module 8

3 Pin Assignment **11**

4 Simulation Results **12**

5 FPGA Implementation **13**

List of Figures

1 Pin Assignment. 11

2 Waveforms in the model-sim simulation 12

3 Waveforms Observed with Oscilloscope 12

4 Transferring Data between transmitter and receiver of same FPGA. 13

5 Transferring Data between two FPGAs. 13

1 Introduction

UART (Universal Asynchronous Receiver/Transmitter) is a serial communication method commonly used for exchanging data between digital systems. Unlike clock-based protocols, UART operates without a shared clock, relying instead on predefined timing agreed upon by both devices. Each data frame is marked by a start bit at the beginning and one or more stop bits at the end, allowing the receiver to correctly interpret the incoming signal.

In this communication scheme, data is initially available in parallel form and is converted into a serial stream by the transmitting unit before being sent through a single line (Tx). The receiving unit reads this serial data through its Rx line and reconstructs it back into parallel format. The speed of transmission is defined by the baud rate, and both ends must use the same baud rate to ensure accurate data transfer.

The aim of this project is to design and implement a UART transceiver using Verilog. The system is first verified through simulation by developing a detailed testbench to validate correct operation under different conditions. After successful simulation, the design is deployed on the DE0-Nano FPGA Development Board.

The implementation includes separate transmitter and receiver modules written in Verilog, along with testbench verification and FPGA configuration. Proper functionality is confirmed by establishing communication between the FPGA and another device via connected Tx and Rx lines, demonstrating successful serial data transmission.

2 Verilog RTL Code

2.1 Transceiver module

```
module uart_transceiver (
    input wire clk,
    input wire rst_n,

    // UART TX Interface
    input wire tx_start,
    input wire [7:0] tx_data,
    output reg tx_busy,
    output reg tx,

    // UART RX Interface
    input wire rx,
```

```
    output reg rx_done,
    output reg [7:0] rx_data
);

parameter CLK_FREQ = 50000000;+

parameter BAUD_RATE = 100;
parameter CLKS_PER_BIT = CLK_FREQ / BAUD_RATE;

localparam IDLE  = 2'b00;
localparam START = 2'b01;
localparam DATA  = 2'b10;
localparam STOP  = 2'b11;

// TX registers
reg [1:0] tx_state;
reg [15:0] tx_clk_count;
reg [2:0] tx_bit_count;
reg [7:0] tx_shift_reg;

// RX registers
reg [1:0] rx_state;
reg [15:0] rx_clk_count;
reg [2:0] rx_bit_count;
reg [7:0] rx_shift_reg;
reg rx_d1, rx_d2;

// Synchronize RX input
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rx_d1 <= 1'b1;
        rx_d2 <= 1'b1;
    end else begin
        rx_d1 <= rx;
        rx_d2 <= rx_d1;
    end
end

// TX
```

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        tx_state <= IDLE;
        tx_clk_count <= 0;
        tx_bit_count <= 0;
        tx_busy <= 0;
        tx <= 1'b1;
        tx_shift_reg <= 8'h00;
    end else begin
        case (tx_state)

            IDLE: begin
                tx <= 1'b1;
                tx_clk_count <= 0;
                tx_bit_count <= 0;

                if (tx_start && !tx_busy) begin
                    tx_busy <= 1'b1;
                    tx_shift_reg <= tx_data;
                    tx_state <= START;
                end
            end

            START: begin
                tx <= 1'b0;

                if (tx_clk_count < CLKS_PER_BIT - 1)
                    tx_clk_count <= tx_clk_count + 1'b1;
                else begin
                    tx_clk_count <= 0;
                    tx_state <= DATA;
                end
            end

            DATA: begin
                tx <= tx_shift_reg[0];

                if (tx_clk_count < CLKS_PER_BIT - 1)
                    tx_clk_count <= tx_clk_count + 1'b1;
```

```
                else begin
                    tx_clk_count <= 0;
                    tx_shift_reg <= {1'b0, tx_shift_reg[7:1]};

                    if (tx_bit_count < 7)
                        tx_bit_count <= tx_bit_count + 1'b1;
                    else begin
                        tx_bit_count <= 0;
                        tx_state <= STOP;
                    end
                end
            end

            STOP: begin
                tx <= 1'b1;

                if (tx_clk_count < CLKS_PER_BIT - 1)
                    tx_clk_count <= tx_clk_count + 1'b1;
                else begin
                    tx_clk_count <= 0;
                    tx_busy <= 1'b0;
                    tx_state <= IDLE;
                end
            end

            default: tx_state <= IDLE;
        endcase
    end
end

// RX
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rx_state <= IDLE;
        rx_clk_count <= 0;
        rx_bit_count <= 0;
        rx_done <= 0;
        rx_data <= 8'h00;
        rx_shift_reg <= 8'h00;
```

```
end else begin
    if (rx_done)
        rx_done <= 1'b0;

    case (rx_state)

        IDLE: begin
            rx_clk_count <= 0;
            rx_bit_count <= 0;

            if (rx_d2 == 1'b0)
                rx_state <= START;
        end

        START: begin
            if (rx_clk_count == (CLKS_PER_BIT - 1) / 2) begin
                if (rx_d2 == 1'b0)
                    rx_clk_count <= rx_clk_count + 1'b1;
                else
                    rx_state <= IDLE;
            end else if (rx_clk_count < CLKS_PER_BIT - 1) begin
                rx_clk_count <= rx_clk_count + 1'b1;
            end else begin
                rx_clk_count <= 0;
                rx_state <= DATA;
            end
        end

        DATA: begin
            if (rx_clk_count == (CLKS_PER_BIT - 1) / 2) begin
                rx_shift_reg <= {rx_d2, rx_shift_reg[7:1]};
                rx_clk_count <= rx_clk_count + 1'b1;
            end else if (rx_clk_count < CLKS_PER_BIT - 1) begin
                rx_clk_count <= rx_clk_count + 1'b1;
            end else begin
                rx_clk_count <= 0;

                if (rx_bit_count < 7)
                    rx_bit_count <= rx_bit_count + 1'b1;
```

```
                else begin
                    rx_bit_count <= 0;
                    rx_state <= STOP;
                end
            end
        end

        STOP: begin
            if (rx_clk_count < CLKS_PER_BIT - 1)
                rx_clk_count <= rx_clk_count + 1'b1;
            else begin
                rx_done <= 1'b1;
                rx_data <= rx_shift_reg;
                rx_clk_count <= 0;
                rx_state <= IDLE;
            end
        end

        default: rx_state <= IDLE;
    endcase
  end
end

endmodule
```

2.2 Test bench module

UART Transceiver testbench Module

```
initial begin
    rst_n = 0;
    tx_start = 0;
    tx_data = 0;

    #100;
    rst_n = 1;
    #100;

    // TEST 1
```

```
$display("Starting TEST 1");
tx_data = 8'd180;
tx_start = 1;
#20;
tx_start = 0;
wait(rx_done);
$display("Finished TEST 1, rx_data = %h", rx_data);

#200;

// TEST 2
$display("Starting TEST 2");
tx_data = 8'd16;
tx_start = 1;
#20;
tx_start = 0;
wait(rx_done);
$display("Finished TEST 2, rx_data = %h", rx_data);

#200;

// TEST 3
$display("Starting TEST 3");
tx_data = 8'd156;
tx_start = 1;
#20;
tx_start = 0;
wait(rx_done);
$display("Finished TEST 3, rx_data = %h", rx_data);

#200;

// TEST 4
$display("Starting TEST 4");
tx_data = 8'd89;
tx_start = 1;
#20;
tx_start = 0;
wait(rx_done);
```

```
    $display("Finished TEST 4, rx_data = %h", rx_data);

    #200;

    // TEST 5
    $display("Starting TEST 5");
    tx_data = 8'd0;
    tx_start = 1;
    #20;
    tx_start = 0;
    wait(rx_done);
    $display("Finished TEST 5, rx_data = %h", rx_data);

    #200;

    // TEST 6
    $display("Starting TEST 6");
    tx_data = 8'd255;
    tx_start = 1;
    #20;
    tx_start = 0;
    wait(rx_done);
    $display("Finished TEST 6, rx_data = %h", rx_data);

    #1000;
    $stop;
end
```

2.3 Top module

```
module de0_nano_top (
    input  wire       CLOCK_50,
    input  wire       KEY0,        // reset, active low
    input  wire       KEY1,        // trigger, active low
    output wire [7:0] LED,
    output wire [6:0] SEG,         // 7-segment (Common Anode)
    output wire       GPIO_0_0,    // TX
    input  wire       GPIO_0_1     // RX
);
```

```
// Clock / Reset aliases
wire clk   = CLOCK_50;
wire rst_n = KEY0;

// UART signals
wire       tx_busy;
wire       tx;
wire       rx_done;
wire [7:0] rx_data;
reg        tx_start;
reg  [7:0] tx_data;

// UART instance
uart_transceiver uart (
    .clk      (clk),
    .rst_n    (rst_n),
    .tx_start (tx_start),
    .tx_data  (tx_data),
    .tx_busy  (tx_busy),
    .tx       (tx),
    .rx       (GPIO_0_1),
    .rx_done  (rx_done),
    .rx_data  (rx_data)
);

assign GPIO_0_0 = tx;
assign LED      = rx_data;

// KEY1 debounce
reg [19:0] debounce_cnt;
reg        key1_stable;
reg        key1_prev;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        debounce_cnt <= 20'd0;
        key1_stable  <= 1'b1;
    end else begin
```

```
        if (KEY1 != key1_stable) begin
            if (debounce_cnt == 20'd999_999) begin   // 20 ms @ 50 MHz
                key1_stable  <= KEY1;
                debounce_cnt <= 20'd0;
            end else
                debounce_cnt <= debounce_cnt + 20'd1;
        end else
            debounce_cnt <= 20'd0;
    end
end

// TX button logic
reg [3:0] test_index;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        tx_start   <= 1'b0;
        tx_data    <= 8'h00;
        test_index <= 4'h0;
        key1_prev  <= 1'b1;
    end else begin
        tx_start  <= 1'b0;
        key1_prev <= key1_stable;

        if ((key1_prev == 1'b1) && (key1_stable == 1'b0) && !tx_busy) begin
            tx_data    <= {4'b0000, test_index};
            tx_start   <= 1'b1;
            test_index <= (test_index == 4'hF) ? 4'h0 : test_index + 4'd1;
        end
    end
end

//7Segment decoder


reg [6:0] seg_raw;

always @(*) begin
    case (rx_data[3:0])
```

```
        4'h0: seg_raw = 7'b1000000; //0
        4'h1: seg_raw = 7'b1111001; //1
        4'h2: seg_raw = 7'b0100100; //2
        4'h3: seg_raw = 7'b0110000; //3
        4'h4: seg_raw = 7'b0011001; //4
        4'h5: seg_raw = 7'b0010010; //5
        4'h6: seg_raw = 7'b0000010; //6
        4'h7: seg_raw = 7'b1111000; //7
        4'h8: seg_raw = 7'b0000000; //8
        4'h9: seg_raw = 7'b0010000; //9
        4'hA: seg_raw = 7'b0001000; //A
        4'hB: seg_raw = 7'b0000011; //b
        4'hC: seg_raw = 7'b1000110; //C
        4'hD: seg_raw = 7'b0100001; //d
        4'hE: seg_raw = 7'b0000110; //e
        4'hF: seg_raw = 7'b0001110; //F
        default: seg_raw = 7'b1111111; // all off
    endcase
end

endmodule
```

3 Pin Assignment

LED[0]	Unknown	PIN_A15	7	B7_N0	3.3-V LVTTL		8mA (default)
LED[1]	Unknown	PIN_A13	7	B7_N0	3.3-V LVTTL		8mA (default)
LED[2]	Unknown	PIN_B13	7	B7_N0	3.3-V LVTTL		8mA (default)
LED[3]	Unknown	PIN_A11	7	B7_N0	3.3-V LVTTL		8mA (default)
LED[4]	Unknown	PIN_D1	1	B1_N0	3.3-V LVTTL		8mA (default)
LED[5]	Unknown	PIN_F3	1	B1_N0	3.3-V LVTTL		8mA (default)
LED[6]	Unknown	PIN_B1	1	B1_N0	3.3-V LVTTL		8mA (default)
LED[7]	Unknown	PIN_L3	2	B2_N0	3.3-V LVTTL		8mA (default)
KEY1	Unknown	PIN_E1	1	B1_N0	3.3-V LVTTL		8mA (default)
KEY0	Unknown	PIN_J15	5	B5_N0	3.3-V LVTTL		8mA (default)
GPIO_0_0	Unknown	PIN_D3	8	B8_N0	3.3-V LVTTL		8mA (default)
GPIO_0_1	Unknown	PIN_C3	8	B8_N0	3.3-V LVTTL		8mA (default)
CLOCK_50	Unknown	PIN_R8	3	B3_N0	3.3-V LVTTL		8mA (default)
SEG[0]	Unknown	PIN_F13	6	B6_N0	3.3-V LVTTL		8mA (default)
SEG[1]	Unknown	PIN_T15	4	B4_N0	3.3-V LVTTL		8mA (default)
SEG[2]	Unknown	PIN_T14	4	B4_N0	3.3-V LVTTL		8mA (default)
SEG[3]	Unknown	PIN_T13	4	B4_N0	3.3-V LVTTL		8mA (default)
SEG[4]	Unknown	PIN_R13	4	B4_N0	3.3-V LVTTL		8mA (default)
SEG[5]	Unknown	PIN_T12	4	B4_N0	3.3-V LVTTL		8mA (default)
SEG[6]	Unknown	PIN_R12	4	B4_N0	3.3-V LVTTL		8mA (default)

Figure 1: Pin Assignment.

4 Simulation Results

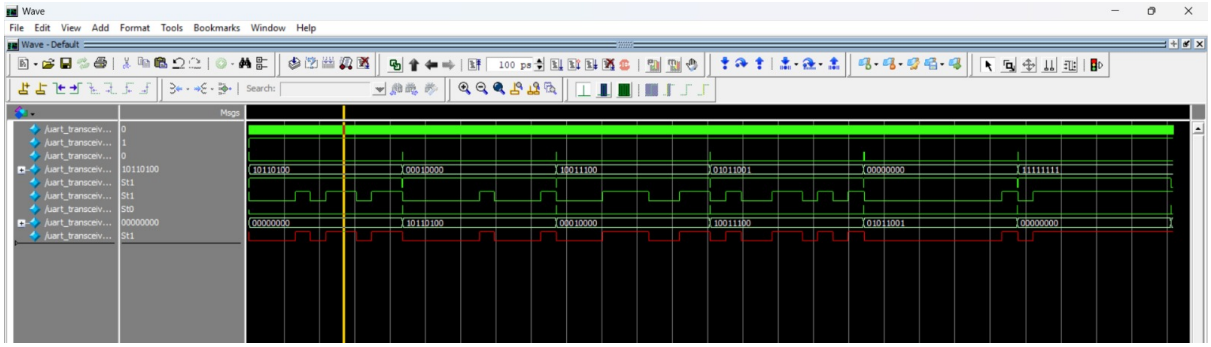

Figure 2: Waveforms in the model-sim simulation

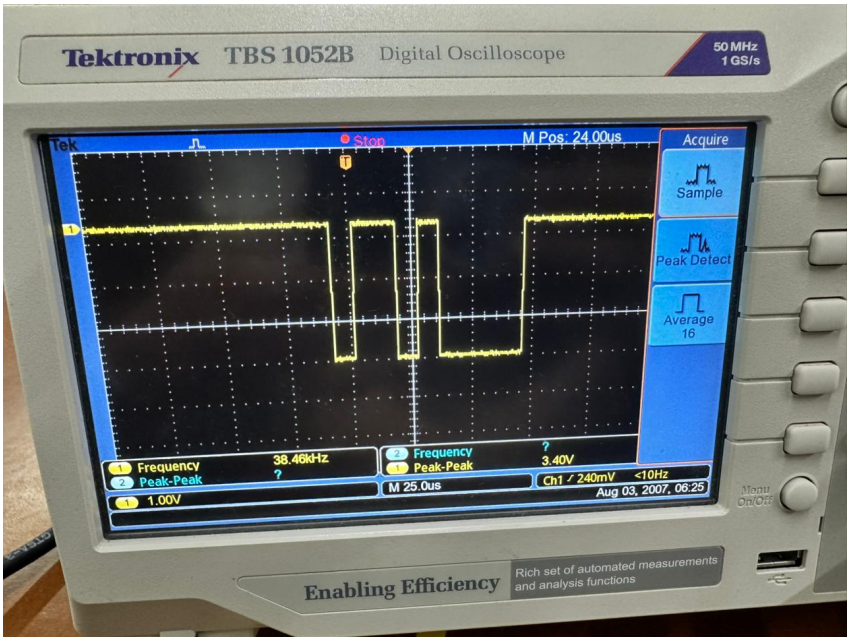

Figure 3: Waveforms Observed with Oscilloscope

5 FPGA Implementation

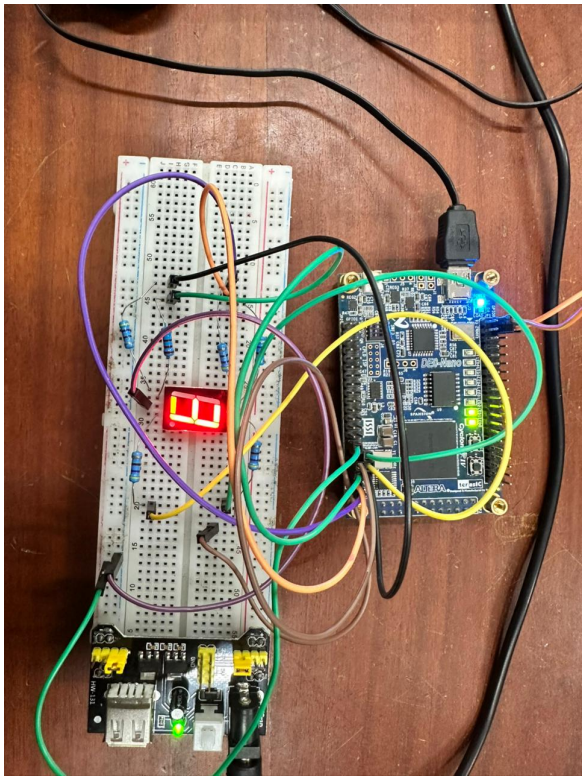

Figure 4: Transferring Data between transmitter and receiver of same FPGA.

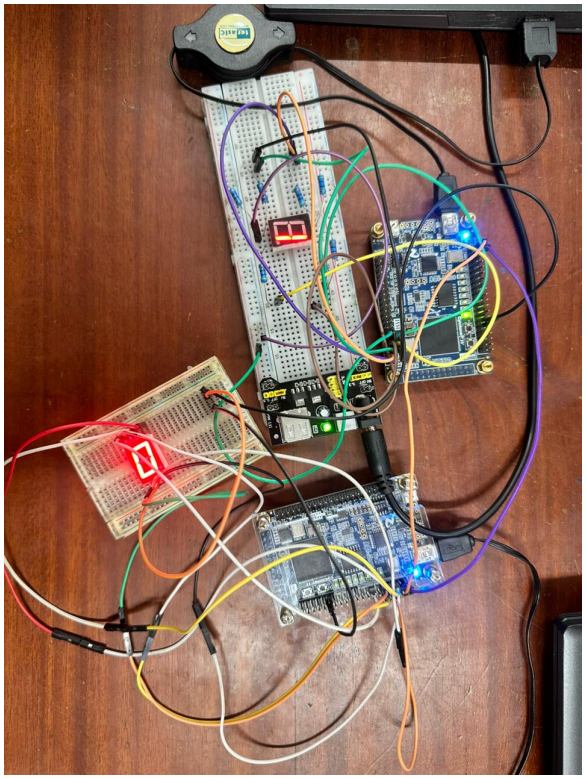

Figure 5: Transferring Data between two FPGAs.